

MusicWall

Project overview and feature guide

A fullscreen music experience built around atmospheric backgrounds, synchronized lyrics, Spotify playback, local music, and theme-specific controls.

Prepared from the current MusicWall codebase

1. What the project is

MusicWall is a React and Vite music player designed as a cinematic fullscreen wall. The player keeps playback controls, lyric presentation, background visuals, browsing, settings, and theme styling in one coordinated interface. Spotify remains the external playback source when connected; local files use the same player controls and queue.

Core principles

Area	Current behavior
Playback	One shared player for Spotify, mock tracks, and imported local audio. Play/pause, previous/next, seeking, mute, and volume are connected to the active source.
Visual layer	Background visuals sit behind the interface, with cinematic wash, vignette, optional grain, motion, and crossfades.
Lyrics	Lyrics are synchronized to playback time and support centered, minimal, cinematic, and Dual Split layouts.
Persistence	Appearance, wallpaper, lyric preferences, local library metadata, recently played data, and custom visuals use the existing local storage/IndexedDB paths.

2. Themes

Themes change the interface design tokens and registered control artwork while keeping playback, lyrics, wallpaper, and settings systems separate.

Theme	Identity and registered assets
Default	Neutral ice and midnight surfaces with standard controls.
Batman	Charcoal and muted gold treatment, bat symbols, Batman Music Library image, and Batman Settings artwork.
Spider-Man	Red, black, and blue web styling, mask play/pause states, spider progress indicator, Spider-Man library and Settings artwork.
Sanrio	Renamed from Hello Kitty while retaining the hello-kitty internal ID for saved preferences. Cherry artwork in the library, pink-heart Settings artwork, Hello Kitty play/pause, pastel bow Previous, skull bow Next, and bow progress thumb.
Sonic	Blue speed styling with ring and streak artwork.

3. Playback and local music

The playback hook and player store keep a single source of truth for the current track, source, elapsed time, duration, playback status, volume, and queue behavior.

Spotify

Spotify authentication uses the existing PKCE flow and current-track/device state. Spotify playback commands are guarded against stale responses and unavailable devices. The interface reports connection and playback limitations instead of pretending controls succeeded.

Local Music

Users can import multiple audio files through the browser file picker or drag and drop. Supported formats include common browser-playable audio such as MP3, WAV, OGG, M4A, AAC, and FLAC. Files stay on the user computer: object URLs are used for playback and metadata/library records are persisted locally. Tracks can be relinked when the original file is unavailable.

Queue behavior

Control	Behavior
Play/pause	Uses the active Spotify or local source.
Previous	Returns to the previous queue item; a local track restarts when appropriate.
Next	Advances through the active local queue and handles the current source.
Progress and volume	Native range inputs remain seekable and volume changes work while paused or playing.
Track end	Local playback advances automatically to the next imported track.

4. Lyrics

Lyrics are fetched and synchronized through the existing lyrics hooks. The active lyric is selected from the current playback position; seeking selects the correct line without animating through skipped lines.

Dual Split mode alternates active lines between stable screen anchors. The current implementation supports upper-left and lower-right composition, restrained opacity and movement transitions, responsive typography, and Motion OFF behavior. Settings include lyric mode, font preset, family, size, weight, spacing, line height, opacity, brightness, glow, blur, animation style, visible lines, position, alignment, and independent singer styles.

5. Background visuals

MusicWall supports saved track visuals, Spotify album artwork fallback, static images, custom image/video uploads, visual presets, and theme-specific panel art. Album artwork can fill the screen with subtle zoom, parallax, gradients, vignette, and crossfading when tracks change. Failed artwork falls back to the existing default background.

Wallpaper controls include source mode, preset, brightness, contrast, saturation, blur, vignette, overlay opacity, color temperature, background opacity, motion intensity, playback speed, looping, inactive pause, fit, and position.

6. Music Library and browsing

Browse Music is a glass sidebar that contains search, recently played items, Made For You content, playlists, local music, album cards, track menus, and visual assignment actions. Each theme inherits the library styling and may provide its own background artwork.

Section	Purpose
Search	Find tracks and library content without leaving the player.
Recently Played	Stores lightweight recent metadata locally, without private object URLs or lyric payloads.
Made For You	Displays the existing curated recommendation area.
Local Music	Lists imported tracks with title, artist/album metadata when available, artwork fallback, relink, remove, and playback actions.
Playlists and cards	Keep the existing card layout, track menus, playlist creation, and queue actions.

Theme-specific library art is applied only to the library panel. Current mappings are Batman artwork, Spider-Man artwork, and the Sanrio image supplied for the Music Library. Other themes retain their normal appearance.

7. Settings control room

Settings is a three-page side panel with Music & Lyrics, Appearance, and Wallpaper navigation. The panel uses a fixed header and tabs with independently scrolling content so the shell stays stable while long sections scroll.

Live previews

The lyric preview uses the active typography variables and shows layout, position, visible lines, style animation, and Dual Split placement. The wallpaper preview uses the selected visual and applies the existing adjustment values without starting a second background engine. Theme cards use the registered theme assets and controls.

Controls and accessibility

Settings controls have theme-aware range fills, numeric values, compact switches, keyboard-visible focus, labeled inputs, tab arrow-key navigation, focus trapping, Escape-to-close, and focus restoration when the panel closes. Motion OFF and reduced-motion preferences remove nonessential movement.

8. Technical structure

Layer	Representative files / responsibility
App composition	src/App.tsx, AppShell.tsx - connects playback, settings, visuals, lyrics, browser, and controls.
Theme registry	src/data/themes.ts - colors, fonts, control assets, decorations, and panel artwork.
Playback	src/hooks/usePlayback.ts, src/store/playerStore.ts, src/hooks/useSpotify.ts - shared playback state and source-specific commands.
Lyrics	src/hooks/useLyrics.ts, LyricsDisplay.tsx, DualSplitLyrics.tsx, lyricStyles.ts - timing, layout, animation, and style variables.
Background	VideoBackground.tsx and AlbumArtBackground.tsx - layered visual loading, fallback, motion, crossfade, and overlays.
Settings	SettingsPanel.tsx, AppearanceSettingsPage.tsx, WallpaperSettingsPage.tsx, settingsValidation.ts - controls, previews, persistence validation.
Testing	Vitest tests cover themes, local playback/library, Spotify sync, lyrics, motion, visuals, album art, and the Settings panel.

9. Current project status

The current codebase includes the feature work completed through the latest request: Sanrio theme naming and player artwork, theme-specific panel backgrounds, local music support, album-art background fallback, Dual Split lyrics positioning, Settings UI polish, and persisted-setting validation.

Verification completed during implementation

Check	Result
Automated test suite	18 test files passed; 62 tests passed in the latest full run.
Production build	Vite and TypeScript production build succeeded.
Local server	Vite dev server responds at http://127.0.0.1:5173/.
Asset serving	Sanrio panel artwork and bow assets returned HTTP 200 from the local server.
Visual limitation	A live browser screenshot pass could not be completed because browser access to localhost was blocked by the automation policy.

10. Known limitations and next checks

The project is functional and tested at the code and build level. A final human visual pass is still useful for checking exact panel cropping, small-window spacing, theme contrast, and animation feel in the browser. Spotify device availability can also vary independently of the app, so live playback checks require an active Spotify device.

End of overview